

7. 공통 메시지 구조 정리

7. 공통 메시지 구조 / 전달 규칙 / 장애 대응 규칙

7-1. 왜 이 정리가 필요한가

지금까지 각 프로세스 역할은 이해했어도, 실제 구현하려면 결국 아래가 통일되어야 한다.

- 메시지는 어떤 형식인가
- ACK는 언제 보내는가
- retry는 언제 하는가
- dedup은 어디서 어떤 기준으로 하는가
- failover는 어떤 조건에서 발동하는가

이게 없으면 각 프로세스가 제각각 움직여서 전체가 맞물리지 않는다.

즉 이번 파트는 **시스템 전체의 공통 규칙**이다.

7-2. 전체 데이터 흐름 다시 요약

가장 먼저 전체 흐름을 한 줄로 다시 잡자.

```
Host Simulator
  → Local Agent
  → Relay R1
  → Relay R2
  → Monitor
```

그리고 제어 흐름은:

```
Controller / UI
  → Host Simulator / Local Agent / Relay R1 / Relay R2 / Monitor
```

즉 데이터 메시지는 위에서 아래로,

제어 메시지는 UI에서 각 노드로 간다.

7-3. 메시지 종류

이 시스템에서 오가는 메시지는 크게 4종류로 보면 된다.

1. EVENT 메시지

목적

실제 상태/장애 이벤트 전달

예

- CPU_SPIKE
- SERVICE_DOWN
- SERVICE_RECOVERED
- LATENCY_HIGH

흐름

- Local Agent → R1 → R2 → Monitor
-

2. ACK 메시지

목적

수신 성공 확인

흐름 예

- R1 → Agent
 - R2 → R1
 - Monitor → R2
-

3. CONTROL 메시지

목적

실험 제어

예

- start
- pause
- reset
- inject fault
- set retry=3
- set timeout=1000

흐름

- Controller / UI → 각 프로세스
-

4. STATUS 메시지 또는 상태 조회 응답

목적

화면 표시용 상태 보고

예

- queue length
- pending ack count
- duplicate dropped
- current host state

흐름

- 각 프로세스 → Controller / UI

7-4. EVENT 메시지 공통 구조

이 프로젝트에서 가장 중요한 건 EVENT 메시지다.

최소 구조는 아래처럼 통일하는 게 좋다.

```
{
  "msg_type": "EVENT",
  "event_id": "evt-host1-000041",
  "seq_no": 41,
  "host_id": "host-1",
  "agent_id": "agent-host-1",
  "event_type": "CPU_SPIKE",
  "severity": "WARN",
  "timestamp": "2026-03-19T10:00:13",
  "payload": {
    "cpu": 95
  }
}
```

각 필드 목적

msg_type

- 메시지 종류 식별
- **EVENT**, **ACK**, **CONTROL** 등

event_id

- 이벤트 고유 ID
- dedup의 핵심 기준

seq_no

- host 단위 순서 번호
- ordering 체크용

host_id

- 어느 host에서 발생했는지

agent_id

- 어느 agent가 생성했는지

event_type

- 어떤 이벤트인지
- 예: CPU_SPIKE, SERVICE_DOWN

severity

- 경고 수준
- 예: INFO, WARN, CRITICAL

timestamp

- 이벤트 생성 시각

payload

- 상세 값
- 예: cpu=95, service=DOWN

7-5. ACK 메시지 공통 구조

ACK도 통일해야 한다.

예시는 아래면 충분하다.

```
{
  "msg_type": "ACK",
  "ack_for": "evt-host1-000041",
  "from_node": "r2",
  "timestamp": "2026-03-19T10:00:14"
}
```

각 필드 목적

ack_for

- 어떤 event_id에 대한 ACK인지

from_node

- 누가 ACK를 보냈는지
- r1, r2, monitor

timestamp

- ACK 생성 시각

7-6. CONTROL 메시지 공통 구조

제어 메시지도 정형화해두면 구현이 편하다.

예:

```
{
  "msg_type": "CONTROL",
  "command": "inject_fault",
  "target": "host_simulator",
  "params": {
    "fault_type": "CPU_SPIKE",
    "duration_sec": 10
  }
}
```

또는:

```
{
  "msg_type": "CONTROL",
  "command": "set_policy",
  "target": "r1",
  "params": {
    "ack_timeout_ms": 1000,
    "max_retry_count": 3,
    "dedup_enabled": true
  }
}
```

7-7. STATUS 응답 구조

UI 표시용 상태도 통일하면 좋다.

예:

```
{
  "node_id": "r1",
  "state": "RUNNING",
  "queue_length": 2,
  "pending_ack_count": 1,
  "retry_total": 4,
  "duplicate_dropped": 3
}
```

7-8. ACK 규칙

이제 중요한 규칙이다.

ACK를 언제 보내는가를 통일해야 한다.

1. Agent → R1 구간

R1은 EVENT를 정상 수신하고 유효성/중복 체크를 마치면 Agent에 ACK 가능

즉 의미는:

- "R1이 받았다"
-

2. R1 → R2 구간

R2는 EVENT를 정상 수신하면 R1에 ACK

즉 의미는:

- "R2가 받았다"
-

3. R2 → Monitor 구간

Monitor는 이벤트를 실제로 처리한 후 R2에 ACK

즉 의미는:

- "Monitor가 기록/처리를 끝냈다"
-

핵심 원칙

ACK는 실제로 그 노드가 처리 가능한 상태로 받아들였을 때 보내야 한다.

즉:

- 네트워크에 도착만 했다고 ACK 아님
 - 최소한 유효성 검증과 기본 수신 처리는 끝나야 ACK
-

7-9. retry 규칙

retry는 ACK가 오지 않을 때 동작한다.

기본 규칙

1. 메시지 전송
 2. pending_ack_table 등록
 3. 타이머 시작
 4. timeout 안에 ACK 오면 완료
 5. ACK 없으면 retry
 6. retry_count 증가
 7. max_retry_count 초과 시 실패
-

retry가 필요한 구간

- R1 → R2
- R2 → Monitor

설계에 따라

- Agent → R1도 retry 가능

초기 설계에서는 **Relay 간 retry**를 중심으로 하는 게 가장 이해하기 쉽다.

retry 관련 파라미터

- `ack_timeout_ms`
- `max_retry_count`

예:

- timeout = 1000ms
 - retry = 3회
-

retry의 의미

retry는 **전달 보장 강화**를 위한 핵심 기능이다.

하지만 그 부작용도 있다.

- 중복 메시지 증가 가능
- queue 적체 유발 가능
- 지연 증가 가능

그래서 dedup과 함께 가야 한다.

7-10. dedup 규칙

dedup은 retry 때문에 생기는 중복 처리를 막는다.

dedup 기준

핵심 기준은 **event_id**다.

같은 `event_id`면 같은 이벤트라고 본다.

dedup이 필요한 위치

- R1
- R2
- Monitor

즉 중간 relay와 최종 sink 모두 dedup이 있어야 한다.

dedup 처리 방식

1. 메시지 수신
 2. `event_id` 확인
 3. cache에 있으면 duplicate
 4. 실제 처리 생략
 5. 필요하면 ACK만 반환
-

dedup cache 관리

무한정 저장하면 안 되므로:

- TTL 기반 만료
- 최근 N개만 유지

중 하나를 쓴다.

초기 구현은 TTL이 단순하다.

예:

- 30초 동안 기억
-

dedup의 의미

dedup은 **at-least-once** 전달의 부작용을 제어하는 장치다.

즉:

- retry는 "손실 방지"
- dedup은 "중복 방지"

둘이 같이 가야 한다.

7-11. ordering 규칙

이 프로젝트에서 ordering은 가장 어려운 축 중 하나다.

그래서 초기에는 **강한 보장보다는 감지 중심**으로 가는 게 맞다.

기준

각 host에서 Agent가 `seq_no` 를 증가시키며 이벤트 생성

예:

- 41
 - 42
 - 43
-

Monitor에서 할 일

- host별 마지막 seq 기록
 - 더 작은 seq가 오면 out-of-order 감지
-

초기 구현 권장

- reorder는 하지 않음
- 단지 "순서 꼬임 발생" 표시만 함

이렇게만 해도 실험 포인트로 충분하다.

7-12. failover 규칙

failover는 "기본 경로 실패 시 대체 경로/대체 정책으로 전환"이다.

어디서 필요한가

대표적으로:

- R1이 R2에 못 보낼 때
 - R2가 Monitor에 못 보낼 때
-

발동 조건

보통 아래 중 하나다.

1. timeout 반복

- ACK가 계속 없음

2. retry 초과

- `retry_count > max_retry_count`

3. heartbeat 상 죽음 판단

- 확장 버전에서 사용 가능

초기 구현은 **retry 초과 시 failover 판단**이 단순하다.

failover 방식 예시

A. alternate target 사용

예:

- R1이 R2 대신 다른 경로로 전송

B. direct sink path

예:

- R1이 직접 Monitor로 보내는 fallback

C. buffer 후 복구 대기

예:

- R2가 Monitor down 시 임시 저장

현재 프로젝트에선 **B나 C가 가장 현실적이다.**

예시 흐름

```
R1 → R2 전송  
ACK 없음  
retry 3회 실패  
failover_enabled=true  
→ direct Monitor path 시도
```

또는

```
R2 → Monitor 전송  
ACK 없음  
retry 실패  
buffer_on_failure=true  
→ 메시지 보관  
→ Monitor 복구 후 재전송
```

7-13. at-most-once / at-least-once와의 관계

이건 프로젝트 핵심 비교 포인트다.

at-most-once 성향

특징:

- retry 거의 없거나 제한적
- 손실 가능
- 중복 적음

구현 느낌:

- timeout 짧게 두지 않음
- 실패 시 그냥 drop 가능
- dedup 부담 적음

at-least-once 성향

특징:

- ACK 없으면 재전송
- 손실 적음

- 중복 가능성 있음

구현 느낌:

- retry 적극 사용
- dedup 필수
- Monitor에서도 idempotent 처리 필요

프로젝트 해석

이 시스템은 기본적으로 **at-least-once + dedup** 쪽이 핵심 실험에 더 적합하다.

왜냐하면:

- retry 보여주기 좋고
- dedup 필요성 설명 가능하고
- 장애 시 전달 성공률 비교가 가능하기 때문이다.

7-14. backpressure / queue 규칙

이건 확장 포인트지만 중요하다.

왜 필요하나

이벤트가 많이 몰리면 relay가 못 버틴다.

예:

- CPU spike
- service down
- latency high
- recovery

가 짧은 시간에 몰리면 queue가 길어진다.

기본 규칙

- queue 길이 모니터링
- 임계치 초과 시 경고
- 신규 메시지 drop 또는 전송 속도 조절

초기 구현 방법

간단하게:

- queue_limit 설정
- 초과 시 low priority drop 또는 warning 표시

확장하면:

- Agent publish 속도 제한
- relay buffer 증가

7-15. 전체 정상 전달 절차

전체를 한 번에 보면 이렇다.

1. Host Simulator가 상태 생성
2. Local Agent가 상태 읽음
3. 이벤트 조건 만족 시 EVENT 생성
4. R1에 전송
5. R1이 수신 후 dedup 검사
6. R1이 Agent에 ACK
7. R1이 R2로 forward
8. R2가 수신 후 dedup 검사
9. R2가 R1에 ACK
10. R2가 Monitor로 forward
11. Monitor가 dedup 검사 후 log/state 갱신
12. Monitor가 R2에 ACK
13. 각 pending_ack 정리
14. 전달 완료

7-16. ACK 유실 시 전체 동작

1. R2가 Monitor로 EVENT 전송
2. Monitor는 실제로 기록 완료
3. Monitor ACK 유실
4. R2 timeout 발생
5. R2 retry
6. Monitor는 같은 event_id 확인
7. duplicate 판단
8. 실제 재기록 안 함
9. ACK만 재전송
10. R2 완료

이게 바로:

- retry 필요성
- dedup 필요성
- idempotent sink 필요성

을 다 보여준다.

7-17. R2 down 시 전체 동작

1. R1이 R2로 EVENT 전송
2. R2 down
3. ACK 없음
4. R1 timeout
5. retry 1, 2, 3
6. max retry 초과
7. failover_enabled 확인
8. true면 alternate path 또는 direct Monitor 시도
9. false면 failure 기록

7-18. 시스템 전체에서 꼭 통일해야 하는 것

실제 구현 전에 아래는 반드시 문서로 못 박아두는 게 좋다.

1. event_id 생성 규칙

예:

- `evt-{host_id}-{seq_no}`

2. seq_no 증가 규칙

- host별 단조 증가

3. ACK 시점

- "수신만"인지
- "로그 기록 완료 후"인지

4. timeout 값

- 예: 1000ms

5. max retry

- 예: 3회

6. dedup cache TTL

- 예: 30초

7. failover 조건

- retry 초과 시

8. failure 처리

- drop / buffer / alternate path 중 무엇인지

이게 없으면 팀 구현에서 서로 다르게 만들 가능성이 높다.

7-19. 전체 공통 구조도

```

EVENT:
Host Simulator
  ↓
Local Agent
  ↓
Relay R1
  ↓
Relay R2
  ↓
Monitor

ACK:
Monitor
  ↑
Relay R2
  ↑
Relay R1
  ↑
(선택적으로 Agent)

```

```

CONTROL:
Controller / UI
  ↳ Host Simulator
  ↳ Local Agent
  ↳ Relay R1
  ↳ Relay R2
  ↳ Monitor

```

7-20. 전체 공통 규칙을 한 문장씩 압축

EVENT

- 상태 변화를 네트워크 전달용 메시지로 표현한다.

ACK

- 각 노드가 메시지를 정상 처리했음을 상위 송신자에게 알린다.

retry

- ACK가 timeout 내 오지 않으면 재전송한다.

dedup

- 같은 event_id는 한 번만 실제 처리한다.

ordering

- seq_no로 순서 이상 여부를 감지한다.

failover

- 기본 경로 전달이 반복 실패하면 대체 경로 또는 대체 정책으로 전환한다.

backpressure

- queue 과부하 시 경고 또는 제한 정책을 적용한다.
-

7-21. 최종 전체 요약

이 프로젝트는 전체적으로 아래 규칙으로 동작한다.

1. **Host Simulator**가 상태/장애를 만든다.
 2. **Local Agent**가 상태를 읽고 EVENT로 바꾼다.
 3. **Relay R1**이 첫 번째로 받아 dedup, ACK, retry, forward를 수행한다.
 4. **Relay R2**가 두 번째로 받아 후단 전달과 buffering/failover를 담당한다.
 5. **Monitor**가 최종적으로 이벤트를 기록하고 현재 상태를 갱신한다.
 6. **Controller / UI**는 장애를 주입하고 정책을 바꾸며 통계를 보여준다.
 7. 전체 전달은 **ACK + timeout + retry + dedup** 규칙으로 신뢰성을 확보한다.
 8. 장애가 반복되면 **failover** 또는 **buffering** 정책으로 대응한다.
-

7-22. 핵심 한 문장 정리

이 시스템은 Agent가 생성한 EVENT를 Relay 노드들이 ACK·timeout·retry·dedup 규칙에 따라 전달하고, 기본 경로 실패 시 failover 또는 buffering으로 대응하여 최종 Monitor까지 신뢰성 있게 전달하는 구조다.

여기까지가 전체 설계 뼈대다.

이제 다음 단계로 가면 가장 자연스러운 건 둘 중 하나다.

- “각 프로세스별 실제 구현 우선순위와 최소 기능 MVP”
- “이걸 코드 파일 단위/모듈 단위로 어떻게 나눌지”